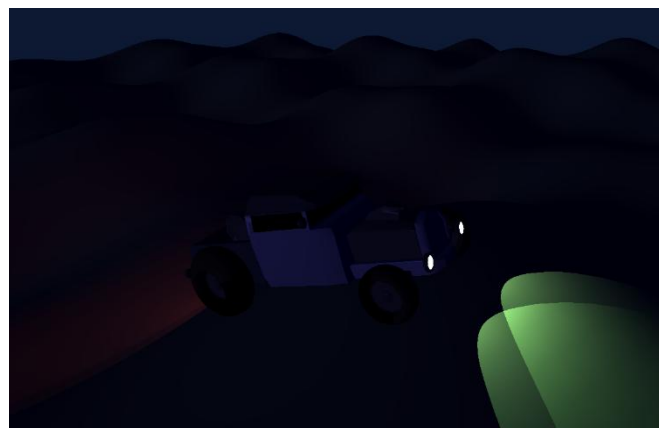
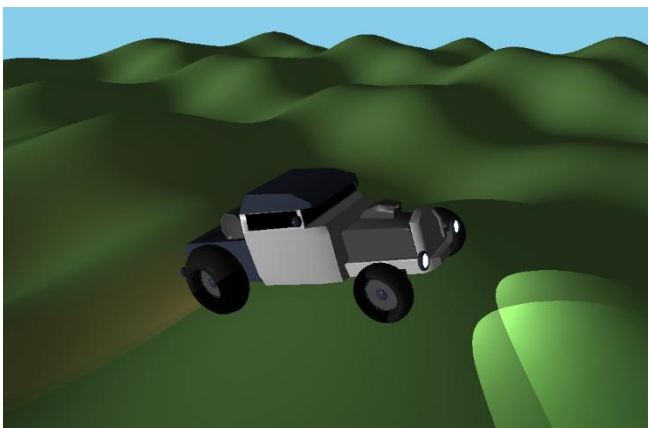


Proseminar Visual Computing Winter Semester 2025

Assignment 4

Hand-out: November 18, 2025

Hand-in: December 01, 2025



Topics

- Shader programming
- Lighting and materials
- Light sources

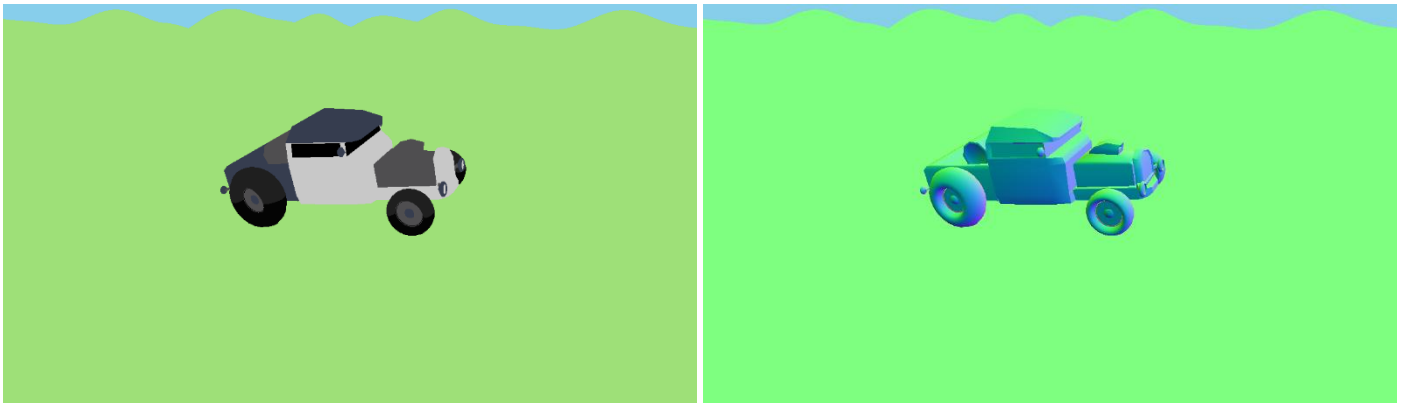
Outline

The goal of the Computer Graphics assignments of the Visual Computing PS is to build an animated pickup. This work is divided into 3 steps. Each step corresponds to a programming assignment. The objective of this assignment is to (a) implement *Blinn-Phong Illumination* for multiple light sources in the *Fragment Shader* via GLSL using material properties defined in the .obj data format and (b) animate a tire deformation in the *Vertex Shader*.

Template code

A template code is provided with this assignment. It loads the pickup truck, ground mesh, and their material properties from .obj/.mtl files. The pickup controls and all animations are already implemented. Currently, two render modes are available (toggle by pressing *R*).

In one mode, the fragment shader sets the fragment color according to the diffuse material, and in the other mode the fragment shader sets the fragment color according to the normals.



Tasks

1. The displacement of the ground is already implemented in the template code. However, to properly implement *Blinn-Phong Illumination*, the surface normals also must be adapted to match the surface. (see **groundCreate**-function; write your calculated normals into the same buffer after the height update; keep the memory layout in mind)

The height of the ground surface is modeled with a sum of periodic waves:

$$H(\mathbf{p}) = \sum W_i(\mathbf{p}),$$

each defined with varying amplitude A , wavelength ω , and direction $\mathbf{d}$:

$$W_i(\mathbf{p}) = A \sin(\omega(\mathbf{p} \cdot \mathbf{d}))$$

Now, for each vertex, compute its corresponding surface normal by taking the partial derivatives of $H(\mathbf{p})$ with respect to x and y (i.e., the 2D coordinates of $\mathbf{p}$).

$$\mathbf{n} = - \begin{pmatrix} 1 \\ \frac{\partial H(\mathbf{p})}{\partial x} \\ 0 \end{pmatrix} \times \begin{pmatrix} 0 \\ \frac{\partial H(\mathbf{p})}{\partial y} \\ 1 \end{pmatrix}$$

Note: By OpenGL convention, the ground is placed in the xz -plane with the y -axis representing the vertical offsets. The above equations are already given in the correct coordinates. Additionally, you should normalize the vector $\mathbf{n}$. The render mode that uses the normals for shading might be helpful for debugging.

2. To better mimic the movement of the pickup, squash the wheels according to the already implemented *squashFactor* (0 = no deformation, 1 = full deformation). There are two different factors, one for the front and one for the back wheels. When moving forward, the back wheels get squashed, and vice versa. The adaptation of the pickup's rotation due to the squashing happens in the *pickup.cpp*-file, you only need to implement the deformation of the wheels themselves. The maximum squash

displacement in the y-direction is defined by the *squashRatio* in percentage. Currently, this value is 1/3. Thus, at *full deformation*, the height of the wheel decreases by one third via squashing.

However, this deformation should happen in the vertex shader, as the calculations can run in parallel on the GPU, which is far more efficient than on the CPU. The easiest way to implement this is to calculate an y-displacement and add to the vertex position after the local deformation is applied, which contains the current wheel rotation.

3. Implement Blinn-Phong illumination in the *Fragment Shader* for a directional light source with the material properties (color and shininess) given in the models. They are loaded from .obj/.mat files as already implemented; see *model.h*.

Find color values of the light source to emulate the lighting condition during day and night. You should be able to switch between them by pressing a key.

4. Next, add point light sources with decaying intensity. There should be *two white lights* for the headlights and *two red lights* for the taillights. The light source positions are provided in the *pickup.cpp* file. Make sure to translate the light source positions as the pickup moves.
5. Change the *point lights* to spotlights. You can implement this without a smooth edge and simply cut the lights power if a fragment is not in the light cone (defined with a direction and a cut-off angle). The following image shows an example of two light cones. You can specify your own setup but make sure that at least two of the lights are *spotlights* (a spotlight with cut-off angle of 180 degrees gives point light).



6. Add the function to toggle the lights. The emission of the pickup lights' materials must be adapted according to the lights being on/off. There is a function *setEmission* for the pickup which toggles all materials with non-zero emissions.

Implementation Remarks

Make sure that your code is clear and readable. Write commentaries when necessary. Your solution should contain a **readme file** with names of the team members, list of keyboard controls, and any explanation that you think is necessary for the comprehension of the code.

Submission and Grading

Submission of your solution is due on December 1th, 2025 (23:59). **Submit the sources** (*i.e.*, only the content of the *src* folder) in a ZIP archive via OLAT. Do not submit the executable and the content of the *build* folder. Do not submit the external dependencies either. Both folder and archive should be named according to the following convention:

Folder: **CGA4_<lastname1>_<lastname2>_<lastname3>**

Archive: **CGA4_<lastname1>_<lastname2>_<lastname3>.zip,**

where <lastname1>, etc. are the family names of the team members. Development in teams of two or three students is requested. Please respect the academic honor code. In total there are 15 marks achievable in this assignment distributed as follows:

- Calculation of the ground normals (**2 marks**)
- Shader based wheel squash animation (**3 marks**)
- A directional light source (sun/moon) (**3 marks**)
- Point light sources attached to the pickup (**4 marks**)
- Spotlight sources (**1 marks**)
- Emission toggle (**1 marks**)
- Code readability, comments, and proper submission: (**1 marks**)

Resources

- Lecture and Proseminar slides as well as code and information are available via OLAT.
- OpenGL homepage
<http://www.opengl.org>
- OpenGL 3.3 reference pages
<https://www.khronos.org/registry/OpenGL/specs/gl/glspec33.core.pdf>
- OpenGL Tutorial for Blinn-Phong Illumination (Phong Shading)
<https://learnopengl.com/Lighting/Basic-Lighting>
<http://www.opengl-tutorial.org/beginners-tutorials/tutorial-8-basic-shading/>
- GL Framework GLFW
<https://www.glfw.org/documentation.html>

Note: Be mindful of employed OpenGL and GLSL versions!